

Loesungen — Blatt 10

Aufgaben: [[numerik/exercises/10/num-exercise-10|Uebung 10]] **PDF:** [[numerik/exercises/10/num-solution-10.pdf|num-solution-10.pdf]] **Quellcode:** numerik/repos/numerik/blatt10/

Inhaltsverzeichnis

- [[#Aufgabe 1 — Ausgleichspolynom $p(x) = a + bx^2$ |Aufgabe 1 — Ausgleichspolynom $p(x) = a + bx^2$]]
 - [[#Aufgabe 2 — Entschärfung eines Signals (Pseudoinverse & TSVD)|Aufgabe 2 — Entschärfung eines Signals (Pseudoinverse & TSVD)]]
-

Aufgabe 1 — Ausgleichspolynom $p(x) = a + bx^2$

Gesucht ist die **lineare Ausgleichsloesung** zum Modell $p(x) = a + bx^2$ mit den 4 Punkten $x = (-2, 0, 1, 2)$, $y = (4, 0, 1, -4)$. Da nur die Monome 1 und x^2 auftreten, ist das Modell **linear in den Parametern** (a, b) .

Aufstellen des ueberbestimmten Systems

Mit $x_i^2 = (4, 0, 1, 4)$ lautet die Designmatrix (Spalten 1 und x^2):

$$A = \begin{pmatrix} 1 & 4 \\ 1 & 0 \\ 1 & 1 \\ 1 & 4 \end{pmatrix}, \quad \begin{pmatrix} a \\ b \end{pmatrix} = c, \quad y = \begin{pmatrix} 4 \\ 0 \\ 1 \\ -4 \end{pmatrix}.$$

Das System $Ac = y$ ist mit 4 Gleichungen und 2 Unbekannten **ueberbestimmt** und i. A. nicht exakt loesbar; gesucht ist das c , das $\|Ac - y\|_2$ minimiert.

Normalgleichungen $A^\top A c = A^\top y$

$$A^\top A = \begin{pmatrix} \sum 1 & \sum x_i^2 \\ \sum x_i^2 & \sum x_i^4 \end{pmatrix} = \begin{pmatrix} 4 & 9 \\ 9 & 33 \end{pmatrix}, \quad A^\top y = \begin{pmatrix} \sum y_i \\ \sum x_i^2 y_i \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Dabei ist $\sum x_i^2 = 4 + 0 + 1 + 4 = 9$, $\sum x_i^4 = 16 + 0 + 1 + 16 = 33$, $\sum y_i = 4 + 0 + 1 - 4 = 1$ und $\sum x_i^2 y_i = 16 + 0 + 1 - 16 = 1$.

Auflösen

Mit $\det(A^\top A) = 4 \cdot 33 - 9 \cdot 9 = 132 - 81 = 51$:

$$a = \frac{1 \cdot 33 - 9 \cdot 1}{51} = \frac{24}{51} = \frac{8}{17} \approx 0.4706, \quad b = \frac{4 \cdot 1 - 9 \cdot 1}{51} = -\frac{5}{51} \approx -0.0980.$$

$$p(x) = \frac{8}{17} - \frac{5}{51} x^2$$

Probe / Residuen

$$p(x_i) = (0.0784, 0.4706, 0.3725, 0.0784), \quad y_i - p(x_i) = (3.92, -0.47, 0.63, -4.08).$$

Residuenquadratsumme $\|Ac - y\|_2^2 \approx 32.63$.

[!warning] Achtung Die Residuen sind gross, weil das **gerade** Modell $a + bx^2$ die Daten nicht gut beschreiben kann: $x_0 = -2$ und $x_3 = 2$ haben denselben x^2 -Wert ($= 4$), aber $y_0 = 4$ und $y_3 = -4$. Das symmetrische Modell kann nur den Mittelwert ($p(\pm 2) \approx 0.078$) treffen — die Ausgleichsrechnung liefert hier trotzdem die *bestmögliche* Anpassung im Sinne der kleinsten Quadrate.

[!tip] Merke Bei einem Modell, das **linear in den Parametern** ist (auch wenn es nichtlinear in x ist, wie hier x^2), bleibt die Ausgleichsrechnung linear: man stellt die Designmatrix A aus den Basisfunktionen auf und löst die Normalgleichungen $A^\top A c = A^\top y$.

Aufgabe 2 — Entschärfung eines Signals (Pseudoinverse & TSVD)

Das Modell $b = Ax$ ist ein **diskretes inverses Problem**: aus dem unscharfen (und verrauschten) Bild $b + \triangle b$ soll das Original x rekonstruiert werden. Die Gauss-foermige Unschärfe-Matrix A ist **extrem schlecht konditioniert**.

Aufbau

```
import numpy as np

N, GAMMA, DELTA = 100, 0.05, 1e-6

def blur_matrix(n=N, gamma=GAMMA):
    c = 1.0 / (gamma * np.sqrt(2 * np.pi))
    i = np.arange(n + 1)
```

```

diff = i[:, None] - i[None, :]
return (c / n) * np.exp(-(diff / (np.sqrt(2) * n * gamma)) ** 2)

def original_signal(n=N):
    x = np.zeros(n + 1)
    x[45:56] = 1.0      # 1 fuer 45..55
    x[60:66] = 0.5      # 1/2 fuer 60..65
    return x

```

Konditionszahl der Matrix: $\text{cond}(A) \approx 2.2 \cdot 10^{18}$ — das Problem ist praktisch singulaer.

(a) Pseudoinverse

```
x_pinv = np.linalg.pinv(A) @ (b + db)    # db = DELTA * randn
```

Ergebnis: $\|x_{\text{pinv}} - x\|_2 \approx 2.9 \cdot 10^8$ — die Rekonstruktion ist **voellig unbrauchbar**. Grund: A^+ teilt durch die winzigen Singulaerwerte σ_i , wodurch der mit $\delta = 10^{-6}$ skalierte Rauschanteil um den Faktor $1/\sigma_i$ (bis $\sim 10^{18}$) **massiv verstaerkt** wird.

(b) TSVD (Truncated SVD) als Regularisierung

Mit der SVD $A = U\Sigma V^\top$ lautet die Pseudoinverse $x = \sum_i \frac{u_i^\top \bar{b}}{\sigma_i} v_i$. Die **TSVD** schneidet alle Singulaerwerte $\sigma_i < \alpha$ ab und summiert nur ueber die “gutmueti- gen” grossen σ_i :

$$x_\alpha = \sum_{\sigma_i \geq \alpha} \frac{u_i^\top (b + \Delta b)}{\sigma_i} v_i.$$

```

U, s, Vt = np.linalg.svd(A)
def tsvd_solve(U, s, Vt, b, alpha):
    mask = s >= alpha
    return Vt.T[:, mask] @ ((U.T @ b)[mask] / s[mask])

```

Rekonstruktionsfehler $\|x_\alpha - x\|_2$ in Abhaengigkeit von $\alpha = 10^k$:

k	α	beruecksichtigter Rang	Fehler $\ x_\alpha - x\ _2$
0	10^0	0	3.54
-1	10^{-1}	14	1.49
-2	10^{-2}	20	1.21
-3	10^{-3}	25	1.06
-4	10^{-4}	29	0.890
-5	10^{-5}	33	0.806
-6	10^{-6}	36	1.49
-7	10^{-7}	40	10.84

k	α	beruecksichtigter Rang	Fehler $\ x_\alpha - x\ _2$
-8	10^{-8}	42	17.97

Das **Optimum liegt bei** $\alpha = 10^{-5}$ (Rang 33): hier ist der beste Kompromiss zwischen Regularisierung und Aufloesung erreicht. (Plots: blatt10/signal_pinv.png, signal_tsvd.png.)

flowchart LR

```
A["alpha gross<br/>(k=0)"] --> B["wenige sigma_i<br/>zu glatt<br/>Fehler 3.5"]
C["alpha optimal<br/>(k=-5)"] --> D["Rang 33<br/>bestes x<br/>Fehler 0.81"]
E["alpha klein<br/>(k=-8)"] --> F["kleine sigma_i<br/>Rauschen verstaerkt<br/>Fehler 18"]
```

[!warning] Achtung Die **naive Pseudoinverse** $A^+(b + \triangle b)$ ist bei schlecht konditionierten inversen Problemen wertlos (hier Fehler $\sim 10^8$), obwohl die Stoerung nur $\delta = 10^{-6}$ betraegt. Die kleinen Singulaerwerte verstaerken das Rauschen um Groessenordnungen.

[!success] Best Practice **TSVD regularisiert** durch Abschneiden kleiner Singulaerwerte. Der Parameter α steuert den Kompromiss: - **zu gross** ($\alpha = 1$): zu viele Komponenten verworfen, Rekonstruktion zu glatt; - **zu klein** ($\alpha = 10^{-8}$): rauschverstaerkende σ_i wieder dabei, Fehler explodiert.

Das ist die diskrete Form des **Bias-Varianz-Kompromisses** bei der Regularisierung schlecht gestellter Probleme.